

PRÁCTICA: Controlador Autónomo de Nave Espacial 2D con ROS2

Robótica: Ciencia de Datos e Inteligencia Artificial

Laura Somolinos

Alejandro Blanco

Héctor Rodríguez

Álvaro Jiménez

INDICE

4.2) DESCRIPCIÓN DEL SISTEMA (pág 3)

4.3) ARQUITECTURA DEL SISTEMA (pág 4)

4.4) DESCRIPCIÓN DEL CONTROLADOR (pág 6)

- MÁQUINA DE ESTADOS
- CÁLCULO DE HEADING DESEADO
- CONVERSIÓN A POTENCIAS DE MOTOR
- ESTRATEGIA DE FRENADO
- MENSAJE DE DEPURACIÓN

4.5) LAUNCH Y PARÁMETROS (pág 10)

- COMANDO COMPLETO DE EJECUCIÓN
- TABLA DE PARÁMETROS

4.6) RESULTADOS EXPERIMENTALES (pág 12)

- TABLA DE RESULTADOS
- CAPTURAS RVIZ
- ANÁLISIS DE RESULTADOS

4.7) ANÁLISIS DE CONCLUSIONES (pág 15)

4.2 Descripción del sistema

El objetivo de esta práctica ha sido desarrollar un controlador autónomo en ROS2 capaz de mover una nave espacial 2D desde una posición inicial hasta un punto objetivo configurable. La nave se simula mediante el paquete `spaceship_sim`, que contiene el nodo físico `ship_simulator` y un nodo de visualización `rviz_publisher`.

La nave tiene una configuración con dos motores, M1 y M2. Cuando ambos motores la tienen la misma potencia, la nave avanza en la dirección de su heading. Cuando las potencias son distintas, aparece un giro que permite cambiar la orientación de la nave. Además, el simulador incluye rozamiento lineal, rozamiento angular y una perturbación de viento configurable. Por tanto, el problema consiste tanto en avanzar hacia el objetivo, como en frenar a tiempo y compensar desviaciones.

La condición de llegada definida por el simulador es:

`distancia_al_target < 2.0` y `velocidad < 0.1 m/s`

Esto significa que no basta con entrar en el círculo del objetivo, sino que la nave también debe llegar con una velocidad suficientemente baja. Esta condición fue una de las principales dificultades de la práctica, ya que un controlador demasiado agresivo hacía que la nave pasara cerca del objetivo pero siguiera orbitando alrededor de él.

Para cumplir los requisitos, se ha creado un paquete propio llamado `spaceship_student_laha`, separado del simulador. Este paquete contiene el nodo principal del controlador, `ship_controller`. También se ha creado el paquete `spaceship_student_laha_msgs`, que contiene el mensaje personalizado `ControlDebug.msg`

El controlador se comunica con el simulador de dos formas principales. Por un lado, se suscribe al topic `/ship_state`, que publica el estado completo de la nave a 20 Hz. A partir de este mensaje se obtiene la posición, velocidad, orientación, target actual y estado de llegada. Por otro lado, en vez de publicar directamente en `/motor_command`, el controlador llama al servicio `/set_motor_power`, de tipo `SetMotorPower`, para cambiar la potencia de los motores. Esta llamada se realiza con `call_async()`, de forma que el nodo no queda bloqueado esperando la respuesta del servicio.

Además, se han añadido dos salidas extra para facilitar la visualización. La primera es el topic `/control_debug`, que publica el mensaje personalizado `ControlDebug`. La segunda es el topic `/controller_markers`, que publica un `MarkerArray` para mostrar en RViz la fase actual del controlador, una flecha con el heading deseado y una flecha hacia el objetivo.

El sistema completo puede lanzarse mediante un fichero launch propio, `student.launch.py`, que permite ejecutar el controlador y, opcionalmente, también el simulador completo usando el parámetro `full_sim`.

4.3 Arquitectura del sistema

La arquitectura final está formada por cuatro paquetes principales:

```
Summary: 4 packages finished [19.3s]
laura@laura:~/ros_ws$ ros2 pkg list | grep spaceship
spaceship_msgs
spaceship_sim
spaceship_student_laha
spaceship_student_laha_msgs
laura@laura:~/ros_ws$ ros2 pkg prefix spaceship_student_laha
/home/laura/ros_ws/install/spaceship_student_laha
laura@laura:~/ros_ws$ ros2 pkg executables spaceship_student_laha
spaceship_student_laha ship_controller
```

Detección de los paquetes ROS2 del sistema

1. **spaceship_msgs**
Contiene los mensajes y servicios comunes del sistema. En este paquete están los mensajes MotorCommand.msg, ShipState.msg y el servicio SetMotorPower.srv.
2. **spaceship_sim**
Contiene el simulador físico de la nave. El nodo ship_simulator integra las ecuaciones de movimiento, aplica el viento, publica /ship_state y ofrece el servicio /set_motor_power. También contiene el nodo rviz_publisher, encargado de publicar la visualización base de la nave en RViz.
3. **spaceship_student_laha**
Es el paquete propio del grupo. Contiene el nodo ship_controller, que implementa el controlador autónomo. Este nodo se suscribe al estado de la nave, calcula las potencias de los motores, llama al servicio de motores y publica información de depuración y visualización.
4. **spaceship_student_laha_msgs**
Contiene el mensaje personalizado ControlDebug.msg, usado para publicar información interna del controlador.

La comunicación entre nodos queda organizada así:

ship_simulator

- publica /ship_state
- publica /ship_target
- recibe /clicked_point
- ofrece servicio /set_motor_power

ship_controller

- se suscribe a /ship_state
- llama a /set_motor_power
- publica /control_debug
- publica /controller_markers

rviz_publisher

- publica marcadores base de la nave en RViz

rviz2

- visualiza la nave, el target y la trayectoria
- visualiza los marcadores del controlador

El simulador se mantiene como motor, mientras que el paquete `spaceship_student_laha` contiene la lógica para que la nave navegue.

El servicio `/set_motor_power` tiene la siguiente estructura:

```
uint8 motor_id
uint8 power
---
bool success
string message
```



```
laura@laura:~$ cd ~/ros_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 service list | grep set_motor
/set_motor_power
laura@laura:~/ros_ws$ ros2 service type /set_motor_power
spaceship_msgs/srv/SetMotorPower
laura@laura:~/ros_ws$ ros2 interface show spaceship_msgs/srv/SetMotorPower
# Comando de potencia para un motor de la nave
# motor_id: 0=ambos, 1=izquierdo (M1), 2=derecho (M2)
uint8 motor_id
# power: 0-100 (%)
uint8 power
---
bool success
string message
```

Comprobación servicio `/set_motor_power` y de la interfaz `SetMotorPower`

El campo `motor_id` indica qué motor se quiere controlar: 0 para ambos motores, 1 para M1 y 2 para M2. El campo `power` indica la potencia entre 0 y 100. La respuesta indica si el comando se ha aplicado correctamente.

El mensaje `ShipState` es la entrada principal del controlador. A partir de este mensaje se leen:

- Posición actual: `x`, `y`
- Orientación: `heading`
- Velocidad lineal: `vx`, `vy`
- Velocidad angular: `omega`
- Potencias actuales: `power_m1`, `power_m2`
- Tiempo transcurrido: `elapsed_time`
- Estado de llegada: `arrived`
- Target actual: `target_x`, `target_y`

```

laura@laura:~/ros_ws$ ros2 topic list | grep ship_state
/ship_state
laura@laura:~/ros_ws$ ros2 topic hz /ship_state
average rate: 19.958
  min: 0.047s max: 0.054s std dev: 0.00178s window: 22
average rate: 20.003
  min: 0.047s max: 0.054s std dev: 0.00162s window: 43
average rate: 19.994
  min: 0.046s max: 0.054s std dev: 0.00179s window: 63
average rate: 19.997
  min: 0.046s max: 0.054s std dev: 0.00168s window: 84
average rate: 20.002
  min: 0.046s max: 0.054s std dev: 0.00165s window: 10
5

```

Comprobación del topic /ship_state, su tipo y su frecuencia cercana a 20 Hz.

Con estos datos, el controlador calcula la acción que debe ejecutar en cada instante.

4.4 Descripción del controlador

El controlador implementado se basa en una estrategia proporcional derivativa sencilla sobre posición y velocidad. La idea principal es no apuntar siempre directamente al objetivo, sino calcular una aceleración deseada que tenga en cuenta tanto el error de posición como la velocidad actual de la nave.

Al principio se probaron estrategias más basadas en fases separadas, como orientar, avanzar, frenar y aproximarse. Sin embargo, el principal problema era que la nave llegaba cerca del objetivo con demasiada velocidad y empezaba a girar alrededor de él. Por este motivo se simplificó el controlador hacia una ley PD, no llega a ser PID ya que no acumulamos el error en el tiempo, se usa un término proporcional al error de posición y un término derivativo asociado a la velocidad de la nave (es decir, proporcional y derivativa, no integral):

$$a_cmd_x = KP \cdot (target_x - x) - KD \cdot vx$$

$$a_cmd_y = KP \cdot (target_y - y) - KD \cdot vy$$

es decir; aceleración deseada en X = KP x error de posición en X - KD x velocidad en X

aceleración deseada en Y = KP x error de posición en Y - KD x velocidad en Y

donde:

- (target_x, target_y) son las coordenadas del objetivo.
- (x, y) son las coordenadas actuales de la nave.
- (vx, vy) son las componentes de la velocidad actual.
- (KP) es la ganancia proporcional de posición.
- (KD) es la ganancia derivativa sobre la velocidad.

En nuestro controlador se han utilizado inicialmente los valores:

$$KP = 0.3$$

$$KD = 1.45$$

La parte proporcional atrae la nave hacia el objetivo, mientras que la parte derivativa actúa como frenado. Si la nave se acerca al objetivo demasiado rápido, el término $(-KDv)$ hace que la aceleración deseada apunte en sentido contrario a la velocidad, ayudando a reducirla antes de entrar en la zona de llegada.

Máquina de estados

Aunque el control real se calcula mediante la ley PD, se mantiene una variable de estado para poder interpretar y visualizar el comportamiento de la nave. Los estados definidos son:

IDLE → THRUST → BRAKE → HOVER → ARRIVED

También se mantiene el estado ORIENT, aunque en la versión actual la orientación se integra dentro del cálculo continuo del control.

La lógica general es:

IDLE: La nave está esperando un target.

THRUST: La nave está lejos del objetivo o necesita avanzar con potencia.

BRAKE: La nave está cerca del objetivo y todavía tiene velocidad significativa.

HOVER: La nave está muy cerca del objetivo o aplicando correcciones pequeñas.

ARRIVED: El simulador ha confirmado que se cumple: distancia < 2 m y velocidad < 0.1 m/s.

El diagrama simplificado de transiciones es:

nuevo target → target válido → IDLE → THRUST → dist < 5 m y vel > 0.20 → BRAKE → dist < 3 m o corrección pequeña → HOVER → msg.arrived = True → ARRIVED

La condición definitiva de llegada no la decide directamente el controlador, sino el simulador mediante el campo arrived del mensaje ShipState.

Cálculo de heading deseado

A partir de la aceleración deseada se calcula el heading hacia el que debería orientarse la nave:

$\text{heading_deseado} = \text{atan2}(a_cmd_y, a_cmd_x)$

Después se calcula el error angular entre el heading deseado y el heading actual:

$\text{error_heading} = \text{heading_deseado} - \text{heading_actual}$

Este error se normaliza al intervalo $[-\pi, \pi]$ mediante la función `angle_diff`.

Conversión a potencias de motor

Una vez calculada la aceleración deseada, se obtiene su módulo:

$$|a_cmd| = \sqrt{a_cmd_x^2 + a_cmd_y^2}$$

Este valor indica cuánta aceleración necesita aplicar la nave. Después se convierte de forma aproximada a una potencia de motor usando la fuerza máxima del simulador:

$$power = (modulo_aceleracion / MAX_THRUST) \cdot 100$$

Como $MAX_THRUST = 3.0$, esta conversión permite pasar de una aceleración deseada a una potencia entre 0 y 100 aproximadamente.

Finalmente, la potencia se limita para evitar valores demasiado bajos o demasiado altos:

$$power = \max(12, \min(max_power, power))$$

El parámetro max_power se puede cambiar desde el launch.

Si el error angular es grande, la nave prioriza girar. En este caso, se aplica potencia a un motor y se deja el otro a cero. Si el error angular es pequeño, la nave avanza con ambos motores, pero aplicando una pequeña corrección diferencial:

Si $error > 0$:

$$M1 = power$$

$$M2 = power - correction$$

Si $error < 0$:

$$M1 = power - correction$$

$$M2 = power$$

La corrección depende del error angular y está limitada por el parámetro $turn_power$.

Estrategia de frenado

El frenado no se implementa como una fase completamente separada, sino que está integrado en la ley PD. La parte derivativa del controlador utiliza la velocidad actual de la nave para reducir la aceleración cuando la nave ya se mueve hacia el objetivo.

Esto tiene una ventaja importante: la nave no espera a estar dentro del círculo del target para frenar, sino que la aceleración deseada va cambiando antes de llegar. Si la nave se desplaza demasiado rápido, la dirección de aceleración calculada puede apuntar parcialmente hacia atrás, reduciendo la velocidad.

Además, si la nave entra en la zona cercana al objetivo y la velocidad es suficientemente baja, el controlador apaga los motores para permitir que el simulador marque la llegada:

$$\text{if } dist < 1.9 \text{ and speed} < 0.10: \text{motores} = 0$$

En las pruebas se observó que el viento podía hacer que la nave tardara más en cumplir la condición de velocidad. Por tanto, se ajustaron las ganancias y la potencia máxima para obtener un equilibrio entre llegar rápido y no pasarse del objetivo.

Mensaje de depuración

Para poder entender mejor el comportamiento del controlador se creó el mensaje ControlDebug.msg. Este mensaje publica en /control_debug:

- Fase actual del controlador.
- Target actual.
- Distancia al objetivo.
- Velocidad de la nave.
- Heading deseado.
- Error angular.
- Aceleración comandada.
- Potencias enviadas a M1 y M2.

```
laura@laura:~/ros_ws$ ros2 interface show spaceship_student_
laha_msgs/msg/ControlDebug
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id

string phase

float64 target_x
float64 target_y

float64 distance_to_target
float64 speed

float64 desired_heading
float64 heading_error

float64 commanded_acceleration

uint8 power_m1
uint8 power_m2
laura@laura:~/ros_ws$ ros2 topic list | grep control_debug
/control_debug
laura@laura:~/ros_ws$ ros2 topic type /control_debug
spaceship_student_laha_msgs/msg/ControlDebug
laura@laura:~/ros_ws$
```

Interfaz del mensaje personalizado ControlDebug y comprobacion del topic /control_debug

Esto nos permitió comprobar, por ejemplo, si la nave estaba en fase THRUST, BRAKE o HOVER, y si las potencias enviadas eran coherentes.

```
header:
  stamp:
    sec: 1780911620
    nanosec: 407247791
  frame_id: world
phase: BRAKE
target_x: 10.0
target_y: 8.0
distance_to_target: 3.9549543679566233
speed: 0.5795879090416466
desired_heading: 0.8091209258970742
heading_error: 0.18797694151242617
commanded_acceleration: 0.3637720721548405
power_m1: 12
power_m2: 6
---
header:
  stamp:
    sec: 1780911620
    nanosec: 457610127
  frame_id: world
phase: BRAKE
target_x: 10.0
target_y: 8.0
distance_to_target: 3.9319759721591825
speed: 0.5774714544734649
desired_heading: 0.8075315251918227
heading_error: 0.18211632112263232
commanded_acceleration: 0.35939672615384555
power_m1: 12
power_m2: 6
```

Ejemplo de publicación real del topic /control_debug durante fase BRAKE

4.5 Launch y parámetros

Para lanzar el sistema completo se ha creado el fichero `spaceship_student_laha/launch/student.launch.py`. Este launch permite ejecutar tanto el simulador como el controlador. El parámetro `full_sim` permite decidir si se lanza también el simulador o solo el controlador.

Comando completo de ejecución

Ejemplo sin viento:

```
ros2 launch spaceship_student_laha student.launch.py \
  target_x:=10.0 target_y:=8.0 \
  max_power:=80 turn_power:=40 \
  wind_strength:=0.0 wind_frequency:=0.0 \
  full_sim:=true
```

Ejemplo con viento máximo:

```
ros2 launch spaceship_student_laha student.launch.py \
  target_x:=10.0 target_y:=8.0 \
  max_power:=80 turn_power:=40 \
  wind_strength:=1.0 wind_frequency:=0.15 \
  wind_seed:=42 full_sim:=true
```

Tabla de parámetros configurables

Parámetro	Valor por defecto	Descripción
<code>target_x</code>	10.0	Coordenada X del objetivo.
<code>target_y</code>	8.0	Coordenada Y del objetivo.
<code>max_power</code>	80	Potencia máxima permitida para los motores desde el controlador.
<code>turn_power</code>	40	Potencia máxima usada para giro diferencial.
<code>wind_strength</code>	0.0	Intensidad del viento en el simulador.
<code>wind_frequency</code>	0.0	Frecuencia de cambio de dirección del viento.
<code>wind_seed</code>	42	Semilla usada para generar viento reproducible.
<code>full_sim</code>	true	Si es true, lanza también el simulador completo.

```

^Claura@laura:~/ros_ws$ ros2 launch spaceship_student_laha s
tudent.launch.py --show-args
Arguments (pass arguments as '<name>:=<value>'):

'target_x':
  Coordenada X del objetivo
  (default: '10.0')

'target_y':
  Coordenada Y del objetivo
  (default: '8.0')

'max_power':
  Potencia máxima permitida para los motores
  (default: '80')

'turn_power':
  Potencia máxima de giro diferencial
  (default: '40')

'wind_strength':
  Intensidad del viento
  (default: '0.0')

'wind_frequency':
  Frecuencia de cambio del viento
  (default: '0.0')

'wind_seed':
  Semilla del viento

```

```

laura@laura:~/ros_ws$ ros2 node list
/rviz2
/rviz_publisher
/ship_controller
/ship_simulator
/transform_listener_impl_5cb858abfe60
/world_broadcaster
laura@laura:~/ros_ws$ ros2 param list /ship_controller
max_power
start_type_description_service
target_x
target_y
turn_power
use_sim_time
laura@laura:~/ros_ws$ ros2 param get /ship_controller target_x
ros2 param get /ship_controller target_y
ros2 param get /ship_controller max_power
ros2 param get /ship_controller turn_power
Double value is: 10.0
Double value is: 8.0
Integer value is: 80
Integer value is: 40

```

Comprobación de los argumentos del launch, nodos lanzados y parámetros del controlador

El launch también permite que el simulador reciba sus parámetros originales, como el objetivo y los parámetros de viento. De esta forma se puede probar fácilmente el controlador con diferentes condiciones sin modificar el código fuente.

Ejemplo con target personalizado

```

ros2 launch spaceship_student_laha student.launch.py \
  target_x:=12.0 target_y:=5.0 \
  max_power:=75 turn_power:=35 \
  wind_strength:=0.4 wind_frequency:=0.10 \
  wind_seed:=42 full_sim:=true

```

4.6 Resultados experimentales

Para evaluar el comportamiento del controlador se han planteado varias ejecuciones con el mismo objetivo, pero con distintas condiciones de viento. En todas las pruebas se debe comprobar si se cumple la condición de llegada:

El objetivo usado para las pruebas principales ha sido: target_x = 10.0 target_y = 8.0

Tabla de resultados

Prueba	Condición de viento	wind_strength	wind_frequency	¿Llega?	Tiempo obtenido (s)	Distancia final (m)	Velocidad final (m/s)	Observaciones
1	Sin viento	0.0	0.0	Sí	21.74	0.72	0.099	La nave llega de forma estable.
2	Viento bajo	0.4	0.10	Sí	47.68	0.81	0.1	La trayectoria se curva ligeramente.
3	Viento medio	0.8	0.12	Sí / A veces tarda más	125.1	1.78	0.098	El viento aumenta el tiempo de estabilización.
4	Viento máximo del enunciado	1.0	0.15	Sí	21.68	0.79	0.1	La nave compensa el viento, sorprendentemente tarda menos que en casos anteriores.

Para completar la tabla se han usado los mensajes de log del simulador, que imprimen una línea del tipo:

LLEGADO ens dist=... speed=...

```
[ship_controller-5] [INFO] [1780935055.997952256] [ship_controller]: [HOVER] pos=(9.1,7.5) dist=1.0m spd=0.16m/s hdg=23° herr=6° t=19.8s
[ship_simulator-1] [INFO] [1780935057.926648684] [ship_simulator]: LLEGADO en 21.74s dist=0.72m speed=0.099m/s
[ship_controller-5] [INFO] [1780935057.952742472] [ship_controller]: ¡LLEGADO! Tiempo: 21.74s
```

También se ha podido comprobar el estado mediante:

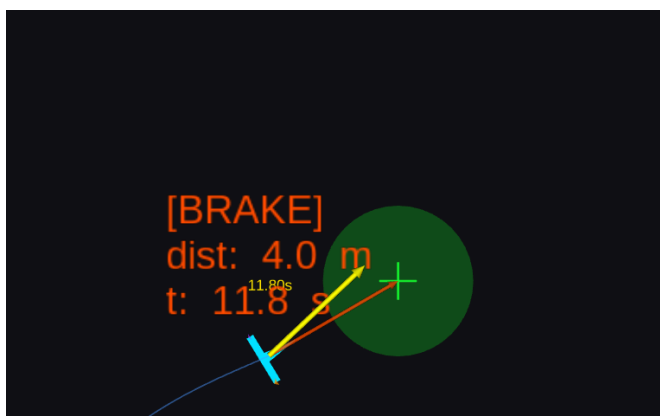
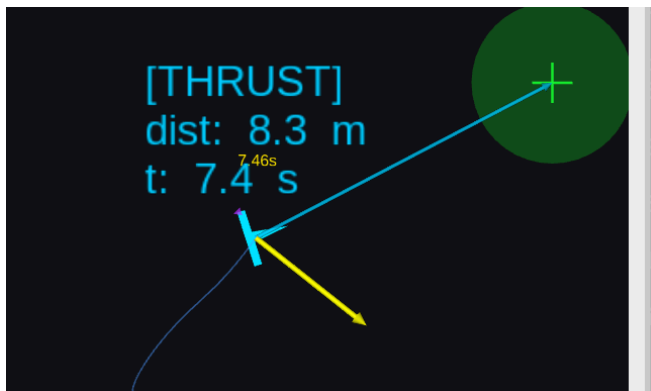
```
ros2 topic echo /ship_state
```

y mediante el topic de depuración:

```
ros2 topic echo /control_debug
```

Capturas de RViz

Se publican tres marcadores en el topic `/controller_markers` mediante `visualization_msgs/MarkerArray`: un texto flotante sobre la nave que muestra la fase activa del controlador (IDLE, ORIENT, THRUST, BRAKE, HOVER o ARRIVED), la distancia al target y el tiempo transcurrido; una flecha amarilla que indica el heading deseado calculado por el controlador PD; y una flecha de color variable que representa el vector de error desde la posición actual hasta el target, cambiando de color según el estado (amarillo en ORIENT, azul en THRUST, naranja en BRAKE, verde en HOVER/ARRIVED).





Análisis de resultados

En las pruebas sin viento la nave alcanza el objetivo de forma estable. La trayectoria no es completamente recta, ya que el controlador va corrigiendo la orientación para ajustar tanto la posición como la velocidad. En la fase inicial predomina THRUST, donde la nave acelera hacia el objetivo. Después, al acercarse, suele aparecer BRAKE, ya que la velocidad todavía es significativa y el controlador empieza a reducirla. Finalmente se pasa a HOVER, donde las potencias son menores y se espera a cumplir la condición de velocidad.

Con viento, la trayectoria se curva más y el tiempo total aumenta. Esto ocurre porque el controlador tiene que compensar la perturbación lateral, por lo que a veces no puede dirigirse exactamente en línea recta al target. Además, cerca del objetivo el viento puede hacer que la velocidad tarde más en bajar por debajo de 0.1 m/s. Aun así, el uso del término derivativo sobre la velocidad ayuda a estabilizar la nave y a evitar que entre en una órbita permanente alrededor del objetivo.

La fase donde se pierde más tiempo suele ser la parte final, entre BRAKE y HOVER, porque la nave ya está dentro o cerca del círculo verde, pero todavía tiene que reducir su velocidad para que el simulador marque arrived.

Como mejora, se podría ajustar dinámicamente las ganancias KP y KD según la distancia al objetivo o añadir un controlador más avanzado que estime el viento y lo compense de forma explícita.

4.7 Conclusiones

En esta práctica hemos desarrollado un sistema ROS2 completo para controlar una nave espacial 2D. La parte más importante ha sido entender que la nave no se comporta como un robot móvil normal, sino que tiene inercia y solo puede acelerar en la dirección de su heading. Por ello, no basta con apuntar siempre al objetivo: también es necesario frenar y controlar la velocidad.

Una de las principales dificultades fue conseguir que la nave no se quedara dando vueltas alrededor del target. Al principio, algunas estrategias de control hacían que la nave llegara cerca del objetivo, pero con demasiada velocidad. Finalmente, usamos una ley de control PD basada en posición y velocidad, que permite anticipar mejor el frenado.

También hemos aprendido la diferencia entre usar topics y servicios en ROS2. Los topics son adecuados para información continua, como `/ship_state`, que se publica a 20 Hz. En cambio, el servicio `/set_motor_power` se usa para enviar una orden concreta al simulador y recibir una respuesta indicando si se ha aplicado correctamente. Además, al usar `call_async()`, el controlador no se bloquea mientras espera la respuesta.

El mensaje personalizado `ControlDebug.msg` ha sido útil para depurar el sistema, ya que permite ver en tiempo real la fase, la distancia, la velocidad, el error angular y las potencias enviadas. Además, los marcadores de RViz facilitan entender visualmente qué está haciendo el controlador.

Como líneas de mejora, se podría implementar un controlador con ganancias adaptativas, usar una estimación explícita del viento, planificar trayectorias más suaves o separar mejor la fase de frenado final. También se podría registrar automáticamente la distancia recorrida y generar gráficas de los experimentos para comparar mejor el rendimiento en distintas condiciones de viento. También pensamos en que esta práctica se podía fusionar con la de otra asignatura, Aprendizaje Automático II en la parte de aprendizaje por refuerzo lo cual presenta algoritmos que podrían ser muy útiles para la llegada de la nave en este entorno.

En conclusión, el sistema cumple los requisitos principales de la práctica: se ha creado un paquete propio, se usa control por servicio, se basa la lógica en `/ship_state`, se ha añadido un mensaje custom funcional, se ha creado un launch parametrizable y se han incorporado marcadores propios en RViz.

El sistema no requiere librerías externas adicionales aparte de ROS2 Jazzy y los paquetes estándar utilizados en la práctica. Todos los paquetes se compilan con `colcon build` desde la raíz del workspace.